

Greedy Algorithms fractional Knapsack

①

$n=7$

$M=15$

Object	1	2	3	4	5	6	7	8
Profits	10	5	15	7	6	18	3	
Weight	2	3	5	7	1	4	1	
P/w	5	1.3	3	1	6	4.5	3	

Consider $\frac{P}{w} \rightarrow$ per kg profit

P/w in descending order

(x_1 , x_2 , x_3 , x_4 , x_5 , x_6 , x_7)

② ① ④ ⑦ ① ③ ⑤

$$15 - 1 = 14 \quad x_5$$

$$14 - 2 = 12 \quad x_1$$

$$12 - 4 = 8 \quad x_6$$

$$8 - 5 = 3 \quad x_3$$

$$3 - 1 = 2 \quad x_7$$

$$2 - 2 = 0 \quad x_2$$

$$\sum x_i w_i \leq M$$

constraint

Objective $\sum x_i p_i$
max

$$\sum x_i w_i \rightarrow 1 \times 2 + \frac{2}{3} \times 3 + 1 \times 5 + 0 \times 7 + 1 \times 1 + 1 \times 4 + 1 \times 1$$

$$2 + 2 + 5 + 0 + 1 + 4 + 1 \Rightarrow 15 \rightarrow \text{Total weight}$$

$$\sum x_i p_i \rightarrow 1 \times 10 + \frac{2}{3} \times 5 + 1 \times 15 + 0 \times 7 + 1 \times 6 + 1 \times 18 + 1 \times 3$$

$$10 + \frac{10}{3} + 15 + 6 + 18 + 3 = 54.6 \rightarrow \text{Total profit}$$

Job sequencing with deadlines

Greedy method.

$n=5$

Jobs	J_1	J_2	J_3	J_4	J_5
Profit	20	15	10	5	1
Deadlines	2	2	1	3	3

Machine $\rightarrow$ 1 unit time for job operation of each job (1 hour)

Objective $\rightarrow$ Jobs must be completed in deadlines giving maximum profit

$0 \quad \underline{J_2} \quad 1 \quad \underline{J_1} \quad 2 \quad \underline{J_4} \quad 3$
 3 slots $\rightarrow$ ② ① $\rightarrow J_2, J_1, J_4 \rightarrow$ execution order
 J_3 cannot be put now

Jobs are already arranged in descending order of profit. Now select the job with highest profit

1) Jobs are not arranged in desc order of profit arrange them and then solve keep the job in slots as per deadlines.

profit $\rightarrow J_2 + J_1 + J_4 = 20 + 15 + 5 = 40$ is max profit in deadline
 sequence $\rightarrow J_2, J_1, J_4$
 J_1, J_2, J_4

3

Job consider	slot assign	solution	Profit
—	—	$\emptyset$	0
J_1	[1, 2]	J_1	20
J_2	[0, 1][1, 2]	J_1, J_2	20+15
J_3	[0, 1][1, 2]	J_1, J_2	20+15
J_4	[0, 1][1, 2]	J_1, J_2, J_4	20+15+5
J_5	[2, 3]	"	"

Job sequencing with deadlines

$n=7$

Assignment

Que

Jobs	J_1	J_2	J_3	J_4	J_5	J_6	J_7
Profit	35	30	25	20	15	12	5
Deadline	3	4	4	2	3	1	2

$$0 \underline{J_4} \quad 1 \underline{J_3} \quad 2 \underline{J_1} \quad 3 \underline{J_2} \quad 4$$

$$20 + 25 + 35 + 30 \Rightarrow \boxed{110}$$

profit

~~sequence~~ J_2, J_1 sequence find.

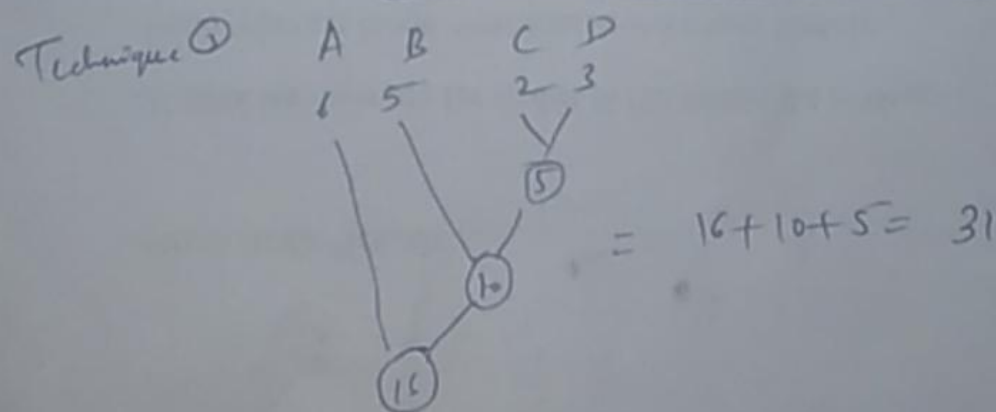
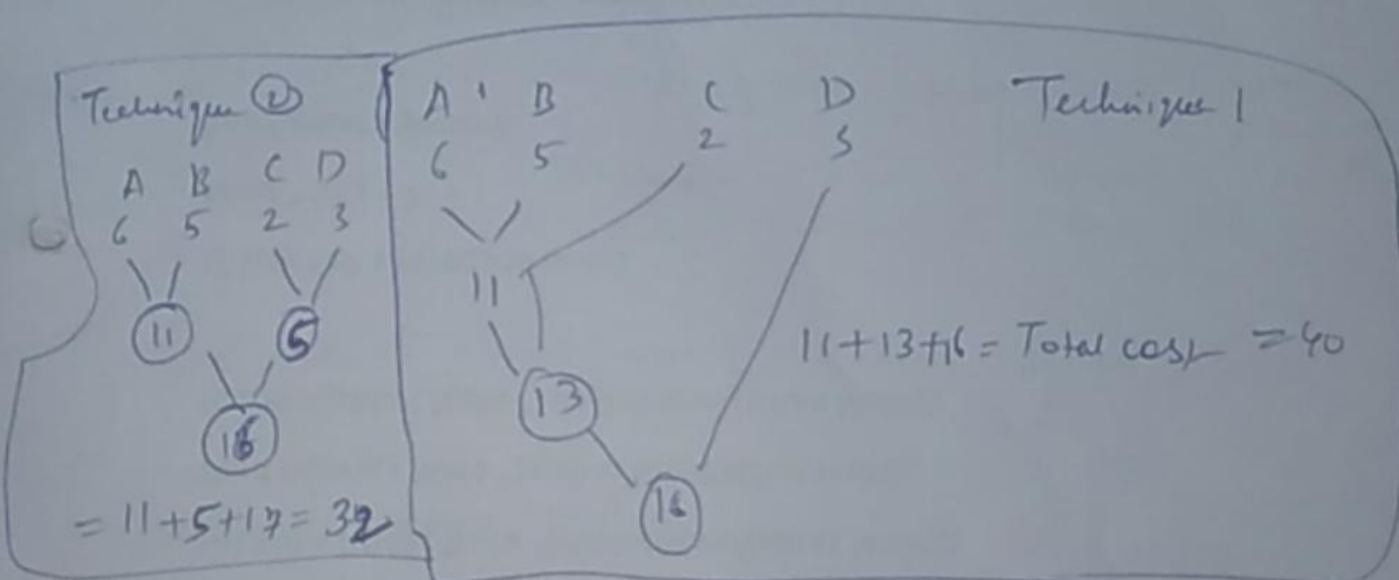
Optimal Merge Pattern

29

A	B	C
3	5	3 (3 < 5)
8	9	5 (5 < 8)
12	11	8 (8 < 11)
20	16	17 (9 < 12)
m	n	11 (11 < 12)
		12 (12 < 20)
		16 (16 < 20)
		20

$\Theta(m+n)$ — Total time taken

List	A	B	C	D
Size	6	5	2	3

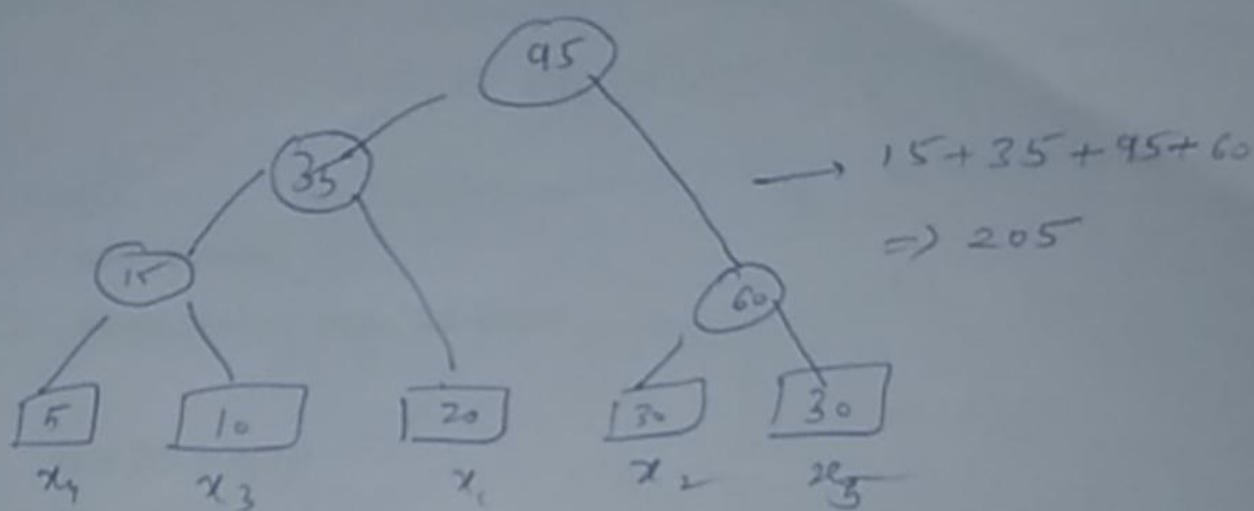


Example problem

list $\rightarrow x_1, x_2, x_3, x_4, x_5$

sizes $\rightarrow 20, 30, 10, 5, 30$

1) Arrange them in increasing order of their size



2) select smallest pairs

$$\begin{array}{ccccccccc}
 3) & 3 \times 5 & + & 3 \times 10 & + & 2 \times 20 & + & 2 \times 30 & + & 2 \times 30 & = & 205 & \text{same as above adding} \\
 & \downarrow & & \downarrow & & \downarrow & & \downarrow & & \downarrow & & & \\
 & 5 \text{ merged} & & 10 \text{ merged} & & 20 \text{ merged} & & 30 \text{ merged} & & 20 \text{ merged} & & & \\
 & 3 \text{ times} & & 3 \text{ times} & & 2 \text{ times} & & 2 \text{ times} & & 2 \text{ times} & & &
 \end{array}$$

$\sum d_i \times x_i = \text{Total cost of merging of all files.}$

Used in Huffman coding.

⑥

Huffman coding

compression technique used for reducing the size of data

message: B C C A B B D D A E C C B B A E D D C C

length = 20

ASCII code = 8 bits

$8 \times 20 = 160$ bits original msg without compression

we can use our own codes instead of ASCII codes

characters	count / frequency		code
A	3	$3/20$	000 → A
B	5	$5/20$	001 → B
C	6	$6/20$	010 → C
D	4	$4/20$	011 → D
E	2	$2/20$	100 → E
	20		→ $20 \times 3 \text{ bits} = 60 \text{ bits}$

Tree ①

Tree ②

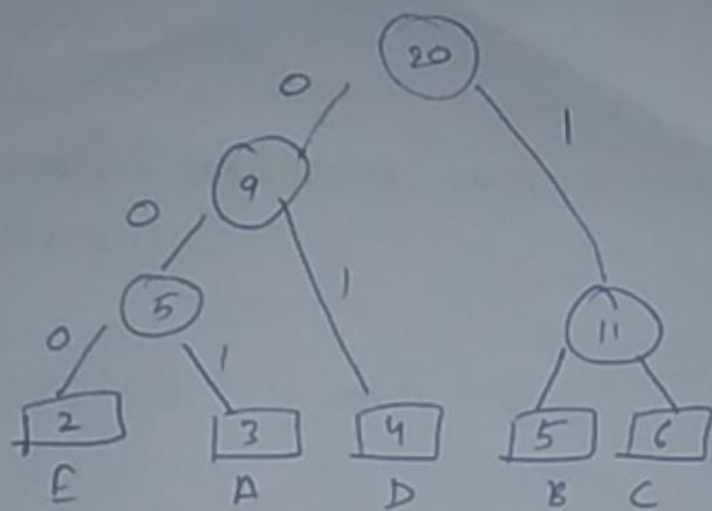
$$\begin{aligned}
 5 \times 8 \text{ bits} &= 40 \\
 5 \times 3 \text{ codes} &= 15 \\
 \downarrow & \\
 40 + 15 &= 55 \text{ bits}
 \end{aligned}$$

msg = 60 bits

Table = $\frac{55 \text{ bits}}{115 \text{ bits}}$

7

char	count	code
A	3	001
B	5	10
C	6	11
D	4	01
E	2	000
		20
		12 bits



1) select 2 minimum

2) left = 0
right = 1

B C C A B B
10 11 11 001 10 10

$$\begin{aligned} \text{msg size} &= 3 \times 3 + 5 \times 2 + 2 \times 6 + 2 \times 4 + 3 \times 2 \\ &= 6 + 10 + 12 + 8 + 6 = 45 \text{ bits} \end{aligned}$$

$$\text{Table size} = 5 \times 8 \text{ bit} + 12 \text{ bit} = 52 \text{ bits}$$

$$\text{Total bits size} = 45 + 52 = 97 \text{ bits}$$

→ can be

found
out
with

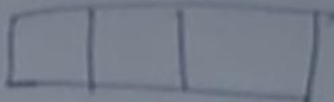
≤ diff
also

msg can be
decoded by bit pattern

Minimum spanning tree I will not

track But I will ask

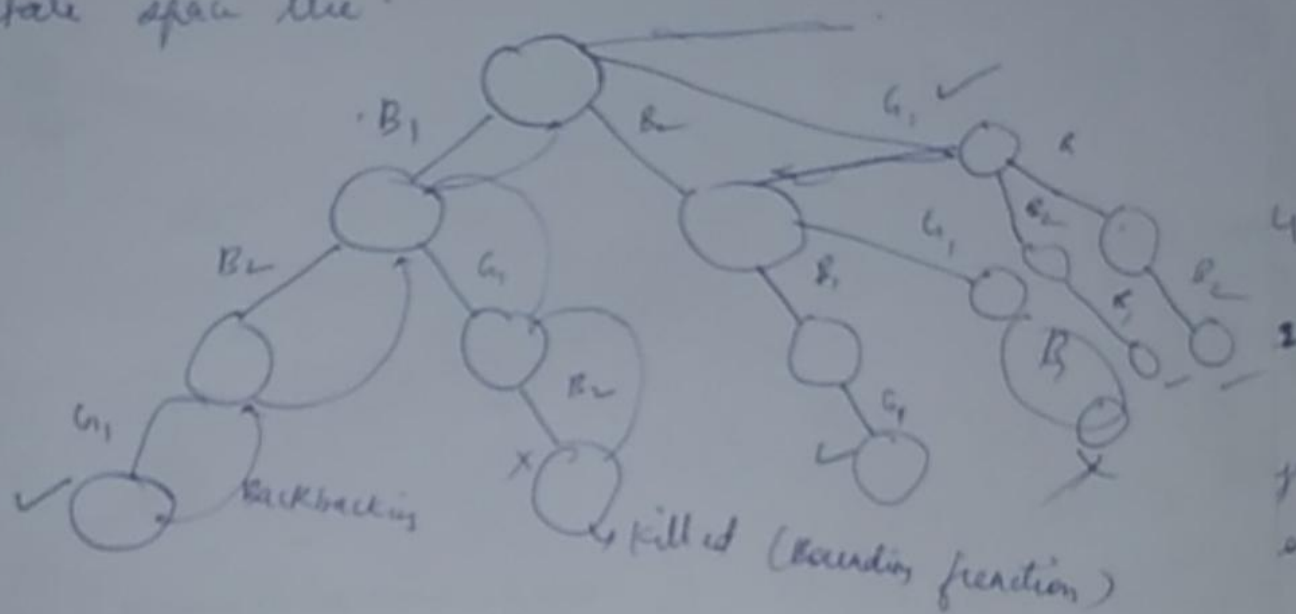
Backtracking
Brute force Approach

B₁ B₂ G₁ 

3 Students 2 Boys 1 Girl 3 chairs
all arrange them in 3! ways $\rightarrow 6!$

Find out all possible arrangements

State space tree



$\rightarrow$ Girl should not sit in middle

$\rightarrow$ All nodes where Girl is in middle that node will be killed.

Backtracking $\rightarrow$ DFS is following
Branch & Recursion $\rightarrow$ BFS is following

State space
tree in back



all levels are
completed and next level
is reached no backtracking
in FB.

Matrix Multiplication (Divide & Conquer) ②

$$A \cdot \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times B \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = C \begin{bmatrix} 1 \\ 1 \end{bmatrix}_{2 \times 2}$$

$\begin{matrix} n & n \\ 2 \times 2 & 2 \times 2 \end{matrix}$

$$C_{ij} = \sum_{k=1}^n A_{ik} * B_{kj}$$

for ($i=0; i < n; i++$) $O(n^3)$

{ for ($j=0; j < n; j++$)

{ $C[i, j] = 0;$

for ($k=0; k < n; k++$)

{ $C[i, j] += A[i, k] * B[k, j];$

}

}

}

using Divide and Conquer

(3)

→ If 2×2 matrices we will directly else we will apply divide and conquer.

$$C_{11} = a_{11} \times b_{11} + a_{12} \times b_{21}$$

$$C_{12} = a_{11} \times b_{12} + a_{12} \times b_{22}$$

$$C_{21} = a_{21} \times b_{11} + a_{22} \times b_{21}$$

$$C_{22} = a_{21} \times b_{12} + a_{22} \times b_{22}$$

→ These operations take constant time

Assume dimensions are in powers of 2 to divide and conquer (example (2×2) , (4×4) , (8×8) , (256×256))

$$A = \begin{array}{c} 4 \times 4 \\ \frac{4}{2} \times \frac{4}{2} \\ \left[\begin{array}{cc|cc} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ \hline a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \\ \hline \boxed{A_{21}} & & \boxed{A_{22}} & \end{array} \right] \end{array}$$

$$B = \begin{array}{c} 4 \times 4 \\ \frac{4}{2} \times \frac{4}{2} \\ \left[\begin{array}{cc|cc} b_{11} & b_{12} & b_{13} & b_{14} \\ \boxed{B_{11}} & & \boxed{B_{12}} & \\ b_{21} & b_{22} & b_{23} & b_{24} \\ \hline b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \\ \hline \boxed{B_{21}} & & \boxed{B_{22}} & \end{array} \right] \end{array}$$

Algorithm MM(A, B, n)

(4)

{ if (n ≤ 2)

{ C = 4 formulas

3

else

{

mid = n/2

→ recursive stack extra space is required

// recursive

MM(A₁₁, B₁₁, n/2) + MM(A₁₂, B₂₁, n/2)

MM(A₁₁, B₁₂, n/2) + MM(A₁₂, B₂₂, n/2)

MM(A₂₁, B₁₁, n/2) + MM(A₂₂, B₂₁, n/2)

MM(A₂₁, B₁₂, n/2) + MM(A₂₂, B₂₂, n/2)

}

$$T(n) = \begin{cases} 1 & n \leq 2 \\ 8T(n/2) + n & n > 2 \end{cases}$$

↓
recursive

→ Time taken 2
add 2 matrices

Applying Master's theorem calls for n/2

a = 8 b = 2 → T(n) = O(n³)

f(n) = n²

To reduce this time complexity it's a research area

Now Strassen has given formula
for this by reducing no of multiplications

(5)

$$P = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$

$$Q = (A_{21} + A_{22}) \cdot B_{11}$$

$$R = A_{11} \cdot (B_{12} - B_{22})$$

7 multiplications
↓
 $7T(\frac{n}{2})$

$$S = A_{22} \cdot (B_{21} - B_{11})$$

$$T = (A_{11} + A_{12}) \cdot B_{22}$$

$$U = (A_{21} - A_{11}) \cdot (B_{11} + B_{12})$$

$$V = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$$

$$C_{11} = P + S - T + U$$

$$C_{12} = R + T$$

$$C_{21} = Q + S$$

$$C_{22} = P + R - Q + U$$

$$T(n) = \begin{cases} 1 & n \leq 2 \\ 7T(n/2) + n^2 & n > 2 \end{cases}$$

$$O(n^{\log_2 7})$$

$$O(n^{2.81})$$

Dynamic programming

→ Optimization over plain recursion.

→ whenever recursive calls are repeated for same i/p we use DP.

→ store the results of subproblems so that we do not have to re-compute them when needed later. This

→ some common problems solved using

DP are → 1) Fibonacci

2) Longest common subsequence

3) Bellman-ford shortest Path

4) Floyd - Warshall

5) Edit distance

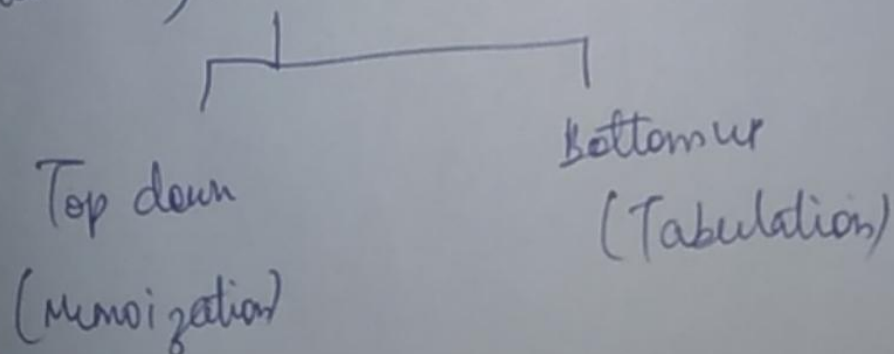
6) Matrix chain Multiplication

- 1) Fibonacci, 2) shortest path
3) Knapsack problem & their time complexity

Dynamic P. is a method used to solve complex problems by breaking them into smaller overlapping subproblems and storing their results to avoid recomputation. It is an optimization technique that transforms recursive solutions with exponential time into efficient ones with polynomial time.

"Remembering the past to solve the future faster".

Two ways to store results



when to use DP

1. Optimal Substructure $\rightarrow$ we use optimal results of subproblems to achieve the optimal result of the bigger problem.

ex problem of finding minimum cost path.

↳ Find the minimum cost path from the source node to each intermediate node

↳ Find the minimum cost path from each intermediate node to destination node

2. Overlapping Subproblems:

The same subproblems are solved repeatedly in different parts of the problem refer to

Overlapping subproblem Property in Dynamic Programming.

Example of DP

Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

Brute force approach

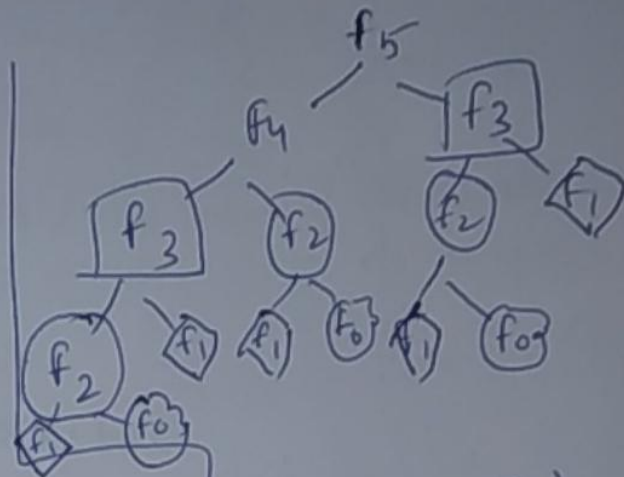
F_1, F_2, F_3 , are repeated

```
int fib(int n)
```

```
{  
  int (n <= 1)
```

```
{  
  return n;
```

```
  return fib(n-1) + fib(n-2);  
}
```



$O(2^n)$

→ Identify subproblems

→ store solutions

→ Build up solutions

→ Avoid recomputation

Memoization $O(n)$ time $O(n)$ space

class S {

int FibRec(int n, ~~vector~~ int[] memo)

{ // Base case

if ($n \leq 1$)

{

return n;

}

if (memo[n] != -1) // To check if O/P already exists

{ return memo[n];

} // calculate and save O/P for future use
memo[n] = FibRec(n-1, memo) + FibRec(n-2, memo);

return memo[n];

}

static int fib(int n)

{ int[] memo = new int[n+1];

Arrays.fill(memo, -1);

return FibRec(n, memo);

}

```

public static void main (String[] args)
{

```

```

    int n = 5;

```

```

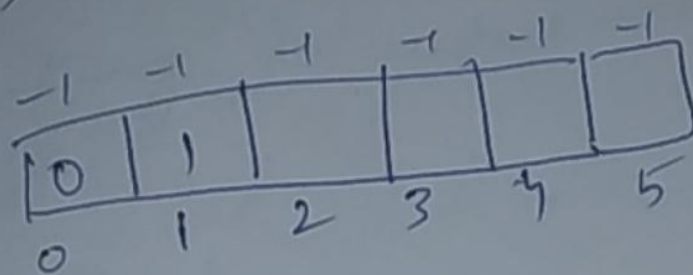
    System.out.println (fib(n));

```

```

}

```



$$\begin{aligned}
 \text{memo}[5] &= \text{FibRec}(4, \text{memo}) + \boxed{\text{FibRec}(3, \text{memo})} \\
 \text{memo}[4] &= \text{FibRec}(3, \text{memo}) + \boxed{\text{FibRec}(2, \text{memo})} \\
 \text{memo}[3] &= \text{FibRec}(2, \text{memo}) + \boxed{\text{FibRec}(1, \text{memo})} \\
 \text{fib.memo}[2] &= \text{FibRec}(1, \text{memo}) + \text{FibRec}(0, \text{memo})
 \end{aligned}$$

~~memo[1] =~~

Tabulation Approach $O(n)$

class S

```
{ static int fibo(int n)
```

```
{ int[] dp = new int[n+1];
```

```
// storing independent values in dp
```

```
dp[0] = 0;
```

```
dp[1] = 1;
```

```
// Using the bottom-up approach
```

```
for (int i = 2; i <= n; i++)
```

```
{ dp[i] = dp[i-1] + dp[i-2];
```

```
}
```

```
return dp[n];
```

```
} public static void main(String[] args) {
```

```
int n = 5;
```

```
System.out.println(fibo(n));
```

```
}
```

```
}
```

```
public static void main (String[] args)
```

```
{  
    int n=5;
```

```
    System.out.println(Fibo(n));  
}
```

```
{  
}
```

memoization

→ Top-down approach

→ stores the results of function calls in a table

→ Recursive implementation

→ Entries are filled when needed

Tabulation

→ Bottom-up approach

→ stores the results of subproblems in table

→ iterative implementation

→ Entries are filled in a bottom up manner from smallest size to the final size

0/1 Knapsack problem Dynamic Programming

item	weight	value/price
1	2	1
2	3	2
3	4	5
4	5	1

$$n=4$$

$$W=8 \text{ kg}$$

$$O(n \times w)$$

Tabulation method $V[i, w] = \max[V[i-1, w], V[i-1, w-w[i]] + v[i]]$

			0	1	2	3	4	5	6	7	8
P	W	0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	2	5	5	6	7	7
6	5	4	0	0	1	2	5	6	6	7	8

$$x = \{0, 1, 0, 1\}$$

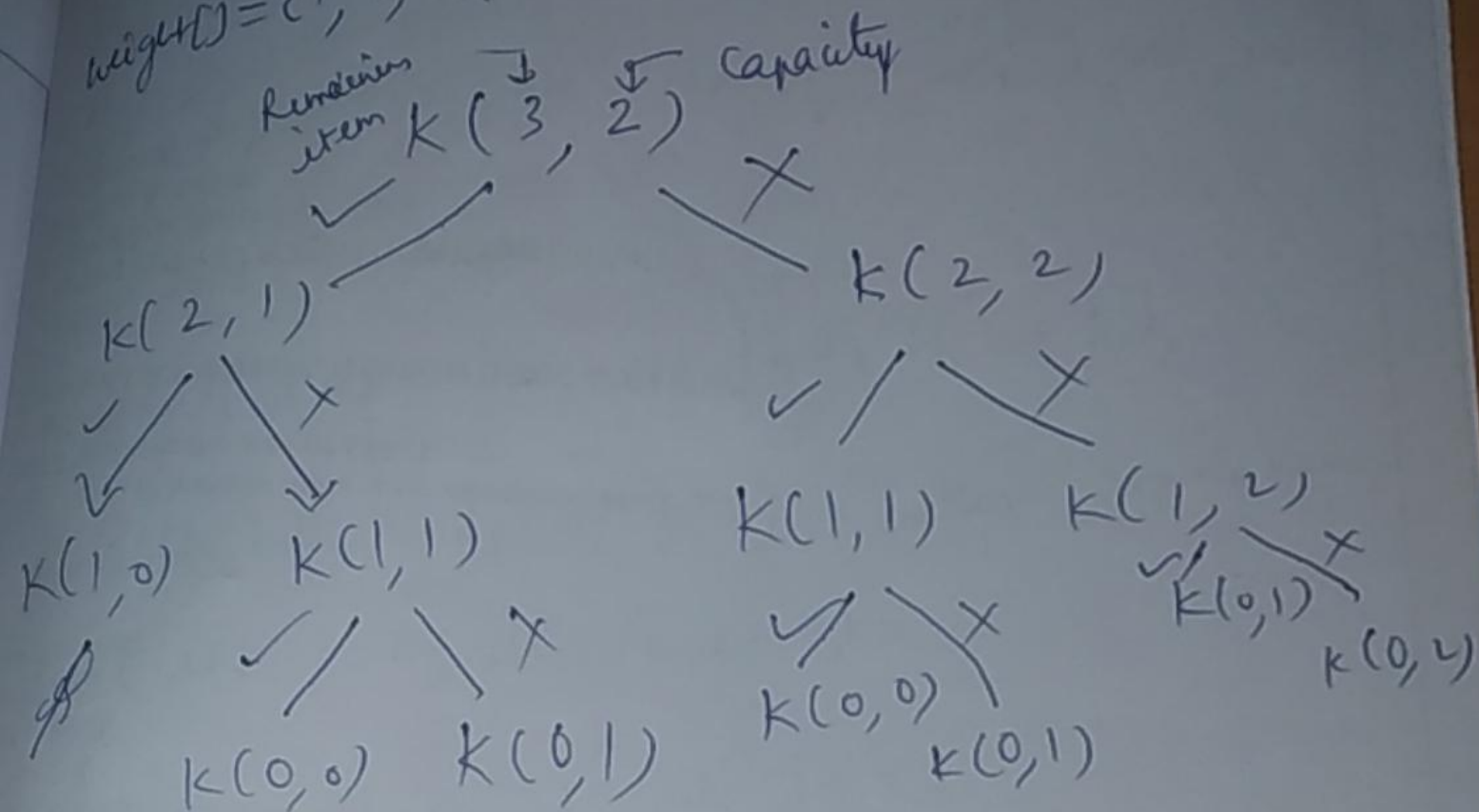
$$x_1, x_2, x_3, x_4$$

$$8-6$$

$$= 2$$

$$2-2=0$$

weight[] = {7, 1, 13} w = 2 value[] = {19, 20, 30}



here $k(1, 1)$, $k(0, 0)$, $k(0, 1)$ are repeated
again and again so we can use dynamic
programming either memoization or tabulation.

```
class S {
```

```
static int knapsack (int W, int[], val, int[] wt)
```

```
{  
    int n = wt.length;
```

```
    int[][] dp = new int [n+1][W+1];
```

```
// build table dp[][] in bottom-up manner
```

```
for (int i=0; i<=n; i++)
```

```
{  
    for (int j=0; j<=W; j++)
```

```
{
```

```
// If there is no item or the Knapsack's  
capacity is 0
```

```
if (i==0 || j==0)
```

```
    dp[i][j] = 0;
```

```
else {
```

```
    int pick = 0;
```

// pick i th item if it does not exceed
the capacity of knapsack

if (wt[i-1] <= j)

pick = val[i-1] + dp[i-1][j - wt[i-1]];

// Don't pick the i th item

int notPick = dp[i-1][j];

dp[i][j] = Math.max(pick, notPick);

}

}

return dp[n][W];

}
public svm (svm —) {

int [] val = {1, 2, 3};

int [] wt = {4, 5, 1};

int W = 4;

System.out.println(knapsack(W, val, wt));

Travelling Salesman Dynamic Programming

	1	2	3	4
1	0	12	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

$$C(S, j) = \min_{i \in S \text{ and } i \neq j} (C(S - \{j\}, i) + d(i, j))$$

$$S = \emptyset$$

$$\text{cost}(2, \emptyset, 1) = d(2, 1) = 5$$

$$\text{cost}(3, \emptyset, 1) = d(3, 1) = 6$$

$$\text{cost}(4, \emptyset, 1) = d(4, 1) = 8$$

$$= 1 \quad \text{cost}(i, S) = \min \{ \text{cost}(j, S - \{j\}) + d(i, j) \}$$

$$\text{cost}(2, \{3\}, 1) = d(2, 3) + \text{cost}(3, \emptyset, 1) = 15$$

$$\text{cost}(2, \{4\}, 1) = d(2, 4) + \text{cost}(4, \emptyset, 1) = 18$$

$$\text{cost}(3, \{2\}, 1) = d[3, 2] + \text{cost}(2, \phi, 1) = 18$$

$$\text{cost}(3, \{4\}, 1) = d[3, 4] + \text{cost}(4, \phi, 1) = 20$$

$$\text{cost}(4, \{3\}, 1) = d[4, 3] + \text{cost}(3, \phi, 1) = 15$$

$$\text{cost}(4, \{2\}, 1) = d[4, 2] + \text{cost}(2, \phi, 1) = 13$$

$$s=2$$

$$\text{cost}(2, \{3, 4\}, 1) = \min \left\{ \begin{array}{l} d[2, 3] + \text{cost}(3, \{4\}, 1) = 29 \\ d[2, 4] + \text{cost}(4, \{3\}, 1) = 25 \end{array} \right\}$$

$$\text{cost}(3, \{2, 4\}, 1) = \min \left\{ \begin{array}{l} d[3, 2] + \text{cost}(2, \{4\}, 1) = 31 \\ d[3, 4] + \text{cost}(4, \{2\}, 1) = 25 \end{array} \right\}$$

$$\text{cost}(4, \{2, 3\}, 1) = \min \left\{ \begin{array}{l} d[4, 2] + \text{cost}(2, \{3\}, 1) = 23 \\ d[4, 3] + \text{cost}(3, \{2\}, 1) = 27 \end{array} \right\}$$

$$s=3$$

$$\text{cost}(1, \{2, 3, 4\}, 1) = \min \left\{ \begin{array}{l} d[1, 2] + \text{cost}(2, \{3, 4\}, 1) = 35 \\ d[1, 3] + \text{cost}(3, \{2, 4\}, 1) = 40 \\ d[1, 4] + \text{cost}(4, \{2, 3\}, 1) = 43 \end{array} \right\} = 35$$

$$C[i, 2] + C[3, 3] + d_{i-1} \times d_2 \times d_3$$

$$C[i, 1] + C[2, 3] + d_{i-1} \times d_1 \times d_3$$

Matrix chain multiplication

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ b_{31} & b_{32} \end{bmatrix}$$

$2 \times 3 \quad 3 \times 2$

$$A \times B = \begin{bmatrix} a_{11} \times b_{11} + a_{12} \times b_{21} + a_{13} \times b_{31} & a_{11} \times b_{12} + a_{12} \times b_{22} + a_{13} \times b_{32} \\ a_{21} \times b_{11} + a_{22} \times b_{21} + a_{23} \times b_{31} & a_{21} \times b_{12} + a_{22} \times b_{22} + a_{23} \times b_{32} \end{bmatrix}$$

No. of columns of A = No. of ~~columns~~ ^{Rows} of B

Total no. of multiplication = $2 \times 3 \times 2 = 12$

dimension of new matrix = 2×2

At $j=3$

$$A_1 \times A_2 \times A_3$$

$2 \quad 3 \quad 4 \quad 2$
 $d_0 \quad d_1 \quad d_2 \quad d_3$

$(A_1 \times A_2) \times A_3$

$2 \quad 3 \quad 4 \quad 2$
 $\downarrow \quad \downarrow$
 $2 \times 3 \times 4 = 24$ multi
 0 multiplication
 Resultant A_3
 $2 \quad 4 \quad 2$

$2 \times 4 \times 2 = 16$
do 2 multi

$24 + 0 + 16 = 40$ mul Total

$C[i, 2] + C[3, 3] + d_0 \times d_1 \times d_2$

$\uparrow$ dimension = 2×2
 $\uparrow$ $\uparrow$ $\uparrow$
 i $k+1$ k j

$A_1 \times (A_2 \times A_3)$

$2 \quad 3 \quad 3 \quad 4 \quad 2$
 $\downarrow \quad \downarrow$
 0 multi
 $2 \times 4 \times 2 = 12$
 Resultant
 A_1
 $2 \quad 3 \quad 3 \quad 2$

$2 \times 3 \times 2 = 12$
do mul $d_1 \quad d_3$

$0 + 24 + 12 = 36$ multiplication

$C[i, 1] + C[2, 3] + d_0 \times d_1 \times d_3$

dimension to Resultant
 2×2

$$C[i, j] = \min_{i \leq k < j} \left\{ C[i, k] + C[k+1, j] + d_{i-1} \times d_k \times d_j \right\}$$

$$A_1 \times A_2 \times A_3 \times A_4$$

do $d_1 \quad d_1 \quad d_2 \quad d_2 \quad d_3 \quad d_3 \quad d_4$

1. $A_1 (A_2 (A_3 A_4))$

2. $A_1 ((A_2 A_3) A_4)$

3. $(A_1 A_2) (A_3 A_4)$

4. $(A_1 (A_2 A_3)) A_4$

5. $((A_1 A_2) A_3) A_4$

Catalan number = $\frac{2^n C_n}{n+1}$

n should be $n-1$

here $n=1$

like $n=4$

that is no. of matrices

so here $n=3$

$$= \frac{2^3 C_3}{3+1} = \frac{6 C_3}{4} = \frac{6 \times 5 \times 4}{3 \times 2 \times 1} = 5$$

multiplication

If $n=5$ $\frac{2^4 C_4}{4+1} = 14$ multiplication

so out of total possibilities we need to find out best possible.

$$C[1,4] = \min$$

$$4-1=3 \quad 1 \leq k < 4$$

difference

$$k=1 \quad A_1 \quad (A_2 \ A_3 \ A_4)$$

$$k=2 \quad (A_1 \ A_2) \quad (A_3 \ A_4)$$

$$k=3 \quad (A_1 \ A_2 \ A_3) \quad A_4$$

Common

$$C[2,4] = \min$$

$$2 \leq k < 4$$

$$(4-2)=2$$

difference

it is becoming lengthy so first find smaller values such that $j-i=1$

		i			
C		1	2	3	4
j	1	0	24	28	58
	2		0	16	36
	3			0	40
	4				0

		K			
		1	2	3	4
j	1				
	2		1		
	3			2	
	4				3

$$A_1 \times A_2 \times A_3 \times A_4$$

$$3 \ 2 \quad 2 \ 4 \quad 4 \ 2 \quad 2 \ 5$$

$$d_0 \ d_1 \quad d_1 \ d_2 \quad d_2 \ d_3 \quad d_3 \ d_4$$

$$C[1,2] = \min_{1 \leq k < 2} \left\{ C[1,1] + C[2,2] + d_0 \times d_1 \times d_2 \right\}$$

$$= 0 + 0 + 3 \times 2 \times 4 = 24$$

$$c[3, 4] = 40$$

$$\rightarrow c[2, 3] = 16$$

$$C[1, 3] = \min$$

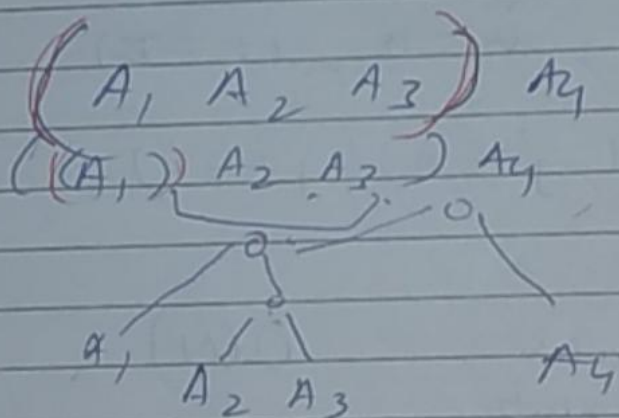
$c[2, 3] = 16$
 $c[1, 3] = \min_{1 \leq k \leq 3} \left\{ \begin{array}{l} k_1 \{ c[1, 1] + c[2, 3] + d_1 \times d_1 \times d_3 \\ k_2 \{ c[1, 2] + c[3, 3] + d_0 \times d_2 \times d_3 \end{array} \right.$

$$C[1, 4] =$$

~~$|K_2| = 86$~~

$$K=2=129$$

$$1c = 3 = (58)$$

$$2 \sin \alpha_1 \cos \alpha_2$$


1 to 4 prearranging
1 to 3 " at 1 }

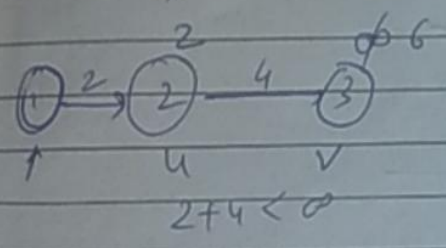
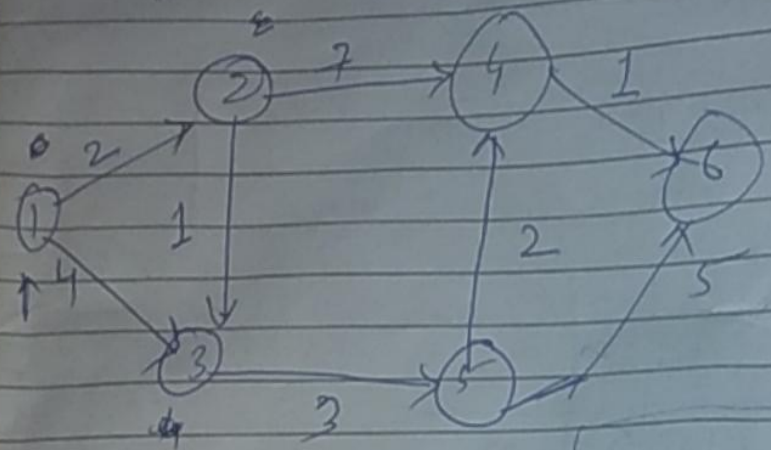
Graph Algorithms

- 1) BFS
- 2) DFS
- 3) Prim's
- 4) Kruskal
- 5) Dijkstra's algorithm

Minimization

Greedy

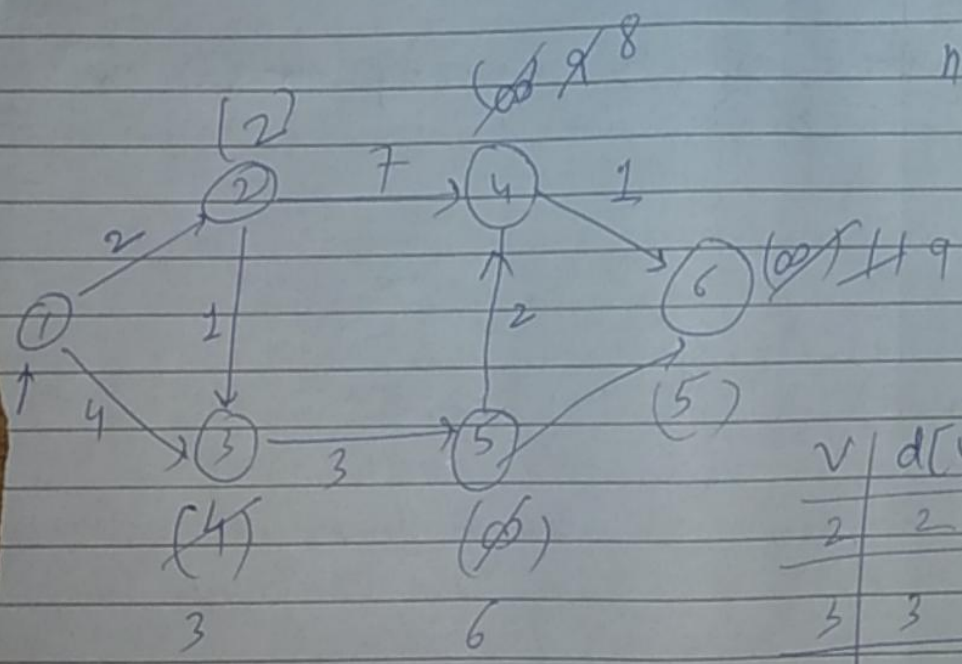
Relaxation



$$\text{if } (d[u] + c(u, v) < d[v])$$

$$d[v] = d[u] + c(u, v)$$

$$n = |V|$$



$$O(n \times n)$$

relaxation

$$O(n^2)$$

v	d[v]
2	2
3	3
4	8
5	6
6	9

①

NP Hard and NP complete

Polynomial Time

Exponential time

Linear search - n 0/1 Knapsack - 2^n Binary search - $\log n$ Travelling SP - 2^n Insertion sort - n^2 sum of subsets - 2^n Merge sort - $n \log n$ Graph colouring - 2^n Matrix Multiplication - n^3 Hamiltonian cycle - 2^n

NP Hard and NP complete are frameworks or guidelines that let us convert (2^n) algo's to polynomial algo's.

1) Try to show the similarities between (2^n) problems so that if one problem is solved another problem can also be solved.

2) Try to solve the problem by relating it.

3) If you can't write deterministic algo's you can write non-deterministic algo's for them.
(polynomial time)

4) Non-deterministic

Algorithm NSearch(A, n, key)

```
{
    j = choice(1, n);
    if (key == A[j])
        write(j);
}
```

```

    success(); — 1
  }
  write(0);
  fail Failure(); — 1
}

```

$O(1)$ Non deterministic

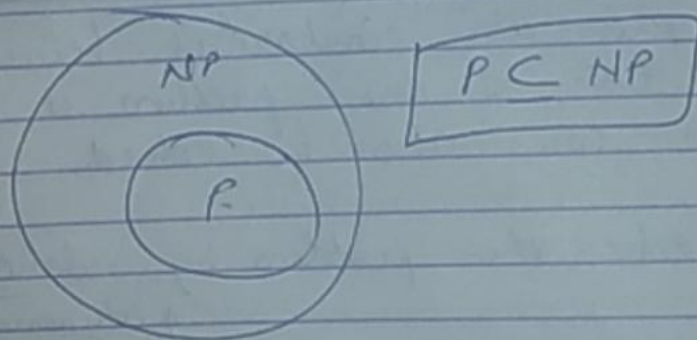
once we know how choice() works it becomes a technique/algo (deterministic)

So for now we are writing N deterministic algo.

Hence we define 2 classes here

P → algorithms that are deterministic P time
example linear search

NP → algorithms that take polynomial time but are Non deterministic.



Base problem → satisfiability problem

CNF satisfiability -

$\wedge$ → conjunction

$\vee$ → disjunction

$$x_i = \{x_1, x_2, x_3\}$$

$\uparrow$ c_1

$$\text{CNF} = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3)$$

$\downarrow$ $\uparrow$ c_2

Propositional formula

c_1 and c_2 are clauses

For what values of x_i this formula is true.

x_1, x_2, x_3
0 0 0

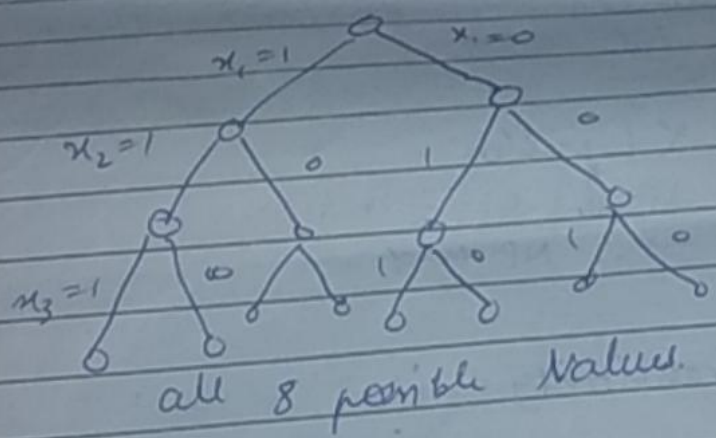
putting all possibilities in formula and check for which values above formula is true.

there are n possibilities

$2^3 \rightarrow 2^n$ for n variables $\rightarrow$ exponential time

so similar to other problems taking 2^n time.

Now we will draw state space tree for this



so we have to prove that if satisfiability is solved in polynomial time all other (2^n) problems can also be solved. (vice versa)

if one is solved all are solved.

NP Hard $\rightarrow$ Among all other (2^n) problems SAT satisfiability is NP Hard because we are considering it as a base problem.

Reduction $\rightarrow$ (2) $\rightarrow$ polynomial

Sat $\propto$ 0/1 Knapsack problem
 $\downarrow$
 satisfiability reduces to $\mathbb{P}_2$
 $\mathbb{P}_1$

convert satisfiability problem to 0/1 Knapsack problem
 as we can say that if 0/1 Knapsack can
 be solved in polynomial time, satisfiability can
 also be solved in polynomial time.

conversion will also take polynomial time.

If satisfiability is reducing 0/1 Knapsack. Then
 Knapsack also becomes NP-Hard.

It means if Sat $\propto$ L problem
 $\downarrow$ $\downarrow$
 NP Hard $\rightarrow \therefore$ NP Hard

Satisfiability Reduction is transitivity

if Sat $\propto$ L_1 and $L_1 \propto L_2$ then Sat $\propto$ L_2 is
 also NP Hard

for satisfiability we have a Non deterministic polynomial
 algo. $\therefore$ So satisfiability is NP Hard as well as
 NP complete

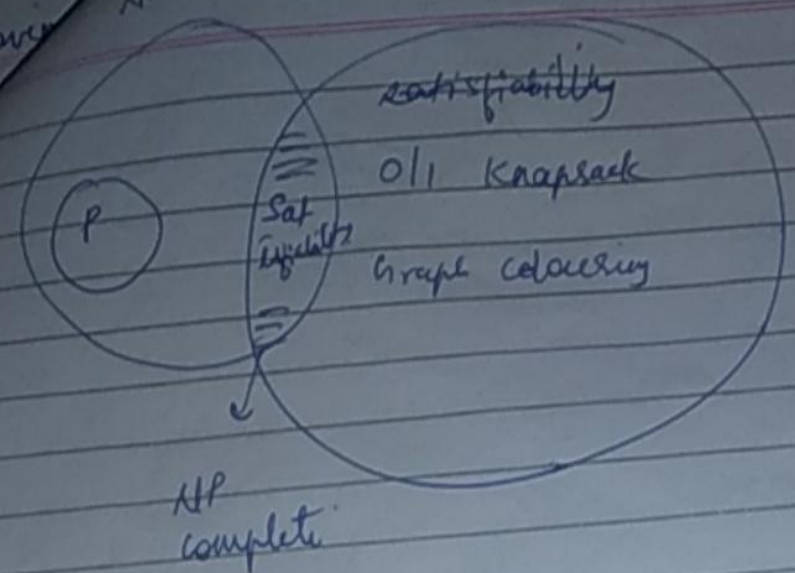
Satisfiability is known NP complete problem.

so if Satisfiability $\propto$ L $\rightarrow$ If we write Non deterministic
 $\downarrow$ $\downarrow$
 NP Hard NP Hard polynomial algo for
 this it becomes NP
 complete.

me - Soniya Nargawse

NP

NP - Hard



P will keep on expanding once deterministic become non-deterministic.

so we should prove that NP contains P
that is $P = NP$ for that we say that

$P \subseteq NP$ → It means P exist but you need to discover it

cook's theorem → Satisfiability will come into P
if it is $P = NP$

Proving a Graph problem as NP Hard

$$P = L_2$$

We have to prove L_2 as NP hard

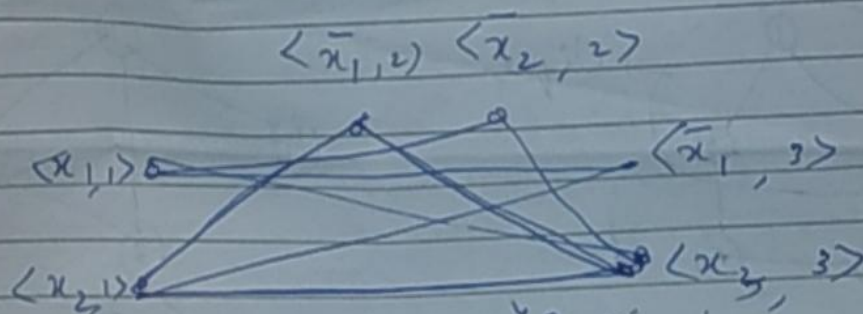
$L_1 \propto L_2$ we need to find L_1 , that is in NP hard. That is satisfiability. satisfiability reduces to CDP

Sat $\propto$ CDP

$$F = (\overset{C_1}{x_1 \vee x_2}) \wedge (\overset{C_2}{\bar{x}_1 \vee \bar{x}_2}) \wedge (\overset{C_3}{\bar{x}_1 \vee x_3})$$

$K=3$

$$F = \bigwedge_{i=1}^K C_i$$



$$G = V = \{ \langle a, i \rangle \mid (a \in C_i) \}$$

if this is solved in polynomial time

Rules for drawing edges

$$1) E = \{ \langle a, i \rangle \langle b, j \rangle \mid i \neq j \text{ and } b \neq \bar{a} \}$$

K clauses

Clique ≤ 3 (max)

because $\bar{x}_1 = T$

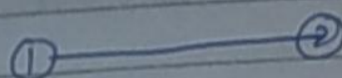
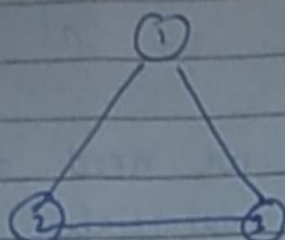
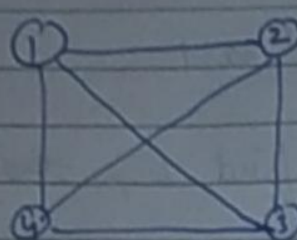
Take all x_1, x_2, x_3 True

$$\begin{matrix} x_1 & x_2 & x_3 \\ 0 & 1 & 1 \end{matrix}$$

$$F = (\overset{0}{x_1} \vee \overset{1}{x_2}) \wedge (\overset{1}{\bar{x}_1} \vee \overset{0}{\bar{x}_2}) \wedge (\overset{1}{\bar{x}_1} \vee \overset{1}{x_3})$$

$= \text{True}$ Final value is true hence proved

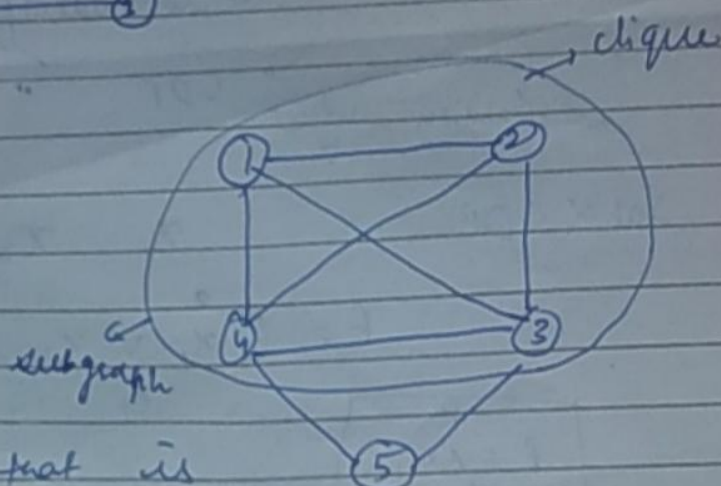
Clique decision problem.



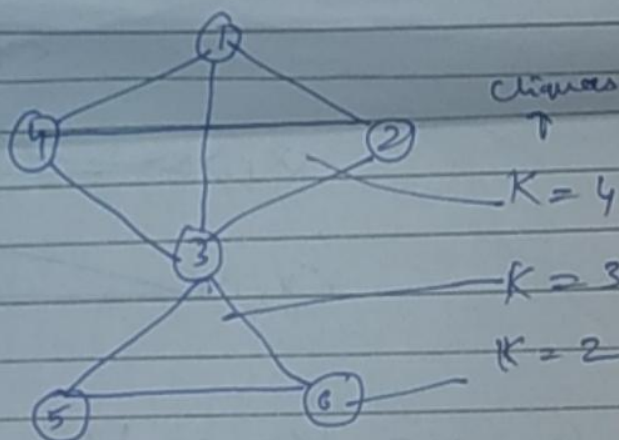
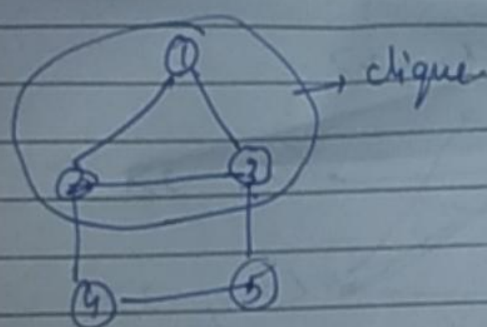
complete graph

$$|V| = n$$

$$|E| = \frac{n(n-1)}{2}$$



subgraph of a graph that is complete is called clique.



Decision problem

Is there a clique of size 3 in this graph?

YES

Problem with True or false

Optimization problem

What is the maximum clique size in the graph?

①

Knuth - Morris - Pratt (KMP) algorithm

Name - $O(mn)$ ^{prefix} ^{suffix}

pattern : abc dabc

prefix : a, ab, abc, abcd

suffix : c, bc, abc, dabc

prefix and suffix are compared and checked whether they are equal or not and then they are matched which takes less time.

π or lps

P_1 : abc dabc ab bf
0 0 0 12 0 12 0

P_2 : abc dabc fab abc
0 0 0 0 12 0 12 3

P_3 : aa bc ada abe
0 1 0 0 1 0 1 2 0 0

P_4 : aaaa ba acd
0 1 2 3 0 1 2 0

↓

string : a b a b c a b c a b a b a b d
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

pattern :

	a	b	a	b	d
j	0	0	1	2	0

$O(m+n)$

preparing
a table

searching
as
alphabet

if ($j = j+1$)

$j = j+1$

$i = j+1$

else

$j = 2$

$j = 0$

$j+1$

Rabin Karp algorithm $\rightarrow$ string matching

Text: $a a a a b$
 $n=6$
 $1+1+1+1=4$

$a=47$

$a=1$

$b=2$

$c=3$

$d=4$

$e=5$

$f=6$

$g=7$

$h=8$

$i=9$

$j=10$

pattern: $a a b$
 $m=3$
 $1+1+2=4 \rightarrow$ hashcode
 $h(p) \rightarrow$ hashfunction

key sliding to next

Rolling

Rabin Karp hash function

Text: $a b c d a b c e$
 $n=8$
 $1+2+3=6$
 $2+3+4=9$

$c=1=5$

pattern: $b c e$
 $p=3$
 $2+3+5=10$

$\rightarrow O(n-m+1)$

no spurious hits

pattern: $d b a$
 $p=3$
 $4+2+1=7$

Text: $c c a c c a e d s a$
 $3+1+3=7$
 $3+1+1=5$
 $3+3+1=7$
 $7+1+5=13$
 $\rightarrow$ 3 matches but pattern does not match
 Collision
 spurious hits
 $O(mn)$

avoid spurious hits

pattern: $d b a$
 $4 \times 10^2 + 2 \times 10^1 + 1 \times 10^0 = 421$

we are using 10

alphabets

till j so far

is 10

$$P[1] \times 10^{n-1} + P[2] \times 10^{n-2} + P[3] \times 10^{n-3}$$